Performance Evaluation of Elasticsearch for Big Data Search in Digital Cultural Heritage Collections

Khin Chan Myae Ko Ko
University of Information Technology
Yangon, Myanmar
khinchanmyaekoko@uit.edu.mm

Zaw Htet Paing
University of Information Technology
Yangon, Myanmar
zawhtetpaing@uit.edu.mm

Kaung Khant Kyaw
University of Information Technology
Yangon, Myanmar
kaungkhantkyaw_2022@uit.edu.mm

Aye Chan Mon
Data Driven Innovation Lab
University of Information Technology
Yangon, Myanmar
ayechanmon@uit.edu.mm

2025 6th International Conference on Advanced Information Technologies (ICAIT) | 979-8-3315-7272-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICAIT68809.2025.11236752

***Abstract*—The digitization of cultural heritage produces massive, heterogeneous datasets that require scalable big data solutions for storage, integration, and search. This paper focuses on optimizing Elasticsearch for archival search tasks, using MongoDB as a reference point for comparison. Data from The Metropolitan Museum of Art and the Harvard Art Museums were integrated through a modular Extract, Transform, Load process, ensuring schema consistency and clean indexing. The tests compared MongoDB regex queries with optimized Elasticsearch configurations, including replacing wildcard queries with match phrase prefix and introducing sharding for distributed execution. Results show that this optimization reduced query latency by up to 82% and improved relevance through linguistic analysis and prefix matching. The findings confirm that proper query design and distributed indexing significantly enhance search speed and scalability for cultural heritage collections, and they can also guide performance tuning in other big-data retrieval domains.**

Keywords—big data analysis, cultural heritage, Elasticsearch, MongoDB, ETL, query optimization, match phrase prefix

I. Introduction

The rapid digitization of cultural heritage produces huge quantities of heterogeneous data involving high-quality images, multilingual text descriptions, and rich relational metadata. This aligns with big data characteristics such as volume in the form of millions of records that demand distributed storage and indexing, variety as diverse file types and schemas that complicate integration, velocity in the form of continuously updated museum data that requires real-time processing, and veracity concerns due to incomplete or inconsistent metadata that affects result accuracy. Effective utilization of this data requires highly scalable storage and retrieval systems that can process high query volumes without losing search relevance.

Traditional relational databases support strong consistency but lack flexibility for handling dynamic schemas. NoSQL databases, such as MongoDB, provide a more flexible schema and can handle diverse data sources, but their native text search may demonstrate underperformance when facing high-load scenarios, which can potentially cause latency in cases of extensive, complex, multi-field queries. Meanwhile, Elasticsearch, a distributed search engine based on inverted indexes, offers scalable full-text retrieval with built-in analyzers for large datasets.

This paper performs a comparative analysis with Elasticsearch as the optimization target and MongoDB as the baseline for search performance. A modular Extract, Transform, Load (ETL) pipeline is used to combine the datasets from the Harvard Art Museums and the Metropolitan Museum of Art. This pipeline guarantees data quality, normalizes schema, and produces formats that are appropriate for indexing and storage. Performance optimization in Elasticsearch includes two techniques: sharding, which partitions an index across nodes to enable parallel queries, and match phrase prefix, which accelerates partial-term matching without costly wildcard.

Experimental results show that these optimizations substantially reduce query latency and improve relevance compared to MongoDB's regex search. Although evaluated on cultural heritage data, the approach generalizes to any domain requiring scalable, text-centric retrieval over heterogeneous datasets.

The remainder of this paper is organized as follows: Section II reviews related work; Section III describes the methodologies; Section IV outlines data acquisition and processing; Section V presents the system architecture; Section VI reports comparative results; and Section VII concludes with findings and recommendations.

II. Related Work

Mass-scale cultural heritage projects increasingly rely on big data infrastructures for managing large-volume, heterogeneous datasets. Gao et al. [1] proposed a Hadoop - based cloud computing model, where HDFS and MapReduce provide distributed storage and processing of vast heritage datasets. Their work highlights the need for fault tolerance and parallelism in heritage informatics, as museums integrate 3D scans and VR into their repositories.

Meanwhile, Big Data Analytics (BDA) has been used in various areas to analyze structured and unstructured data. Islam and Khan [2] outlined BDA's cross-domain promise to span health, education, and public service, but also outlined the key challenges around data quality, infrastructure disparity, and ethical issues related to data governance. These factors are relevant for archival systems managing sensitive or culturally significant content.

Elasticsearch has become a leading solution for near real-time, high-throughput search and analytics at big data scale, particularly in monitoring, indexing, and log analysis. Gandhi and Prasad [3] explored a few best practices related to Elasticsearch such as filter-based queries, index mapping

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.

design, and caching mechanism which significantly increase the performance of a query and thus reducing latency. In recent studies, machine learning integration has enabled predictive optimization of query structures.

Ahir and Shaikh [4] evaluated the Elastic Stack (Elasticsearch, Logstash, Filebeat, Kibana) for anomaly detection and web monitoring, finding strong performance in multi-metric outlier detection but weaker results for single-metric ML. They also noted that advanced ML requires paid tiers and that Kibana's visualization is limited to Elasticsearch data, prompting interest in alternatives such as Grafana or Knowi.

Beyond search engines, studies have also examined NoSQL databases as big data solutions. Singh et al. [5] compared MongoDB with document-based MySQL for CRUD-heavy workloads and found MongoDB superior in insert and update performance due to its mature replication and sharding mechanisms. Their work reinforces MongoDB's suitability for managing semi-structured data efficiently across distributed environments.

However, most prior studies analyze storage scalability or search efficiency separately. Few directly compare document stores and search engines under unified datasets. This paper addresses that gap by benchmarking MongoDB and Elasticsearch with harmonized museum data, showing how query-level optimizations such as match phrase prefix and sharding improve both latency and retrieval relevance in large-scale heritage collections.

III. Methodologies

A. Big Data Analysis (BDA)

BDA refers to extracting insights from large, complex datasets through scalable storage, processing, and analysis techniques. In this work, artifact records from multiple museums were cleaned, normalized, and indexed through a modular ETL pipeline as shown in Fig. 1. This integration addresses the five Vs of big data such as volume, variety, velocity, veracity, and value by ensuring consistent schema mapping, distributed indexing, and high-relevance retrieval.

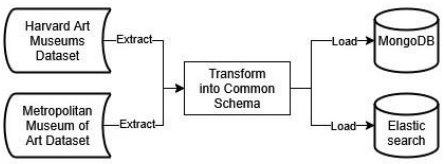

Fig. 1. ETL pipeline

B. MongoDB

MongoDB, a NoSQL document database, was used as the performance benchmark for search due to its flexible schema model for storing heterogeneous and dynamic artifact metadata. It was used for complex, irregular records without pre-defined schemas and is best suited to the integration of heterogeneous cultural heritage datasets.

C. Elasticsearch

Elasticsearch, based on an inverted index, was configured for low-latency, full-text search across large datasets. Custom mappings and analyzers (e.g., porter_stemmer) were used to enhance recall and relevance. Multiple descriptive fields were aggregated into composite searchable fields. Query efficiency was improved by replacing dynamic mappings and wildcard searches with static mappings and match phrase prefix queries that fully utilize the inverted index.

D. ETL (Extract, Transform, Load)

ETL constitutes a standardized data processing pipeline encompassing extraction of raw artifact data, transformation including schema normalization, cleansing and restructuring, followed by loading data into target storage systems. This way, metadata consistency is preserved and best indexing for searching is supported. The ETL pipeline was a critical mechanism to address heterogeneity and inconsistency between different museum data sources during the research. Through schema mapping, management of null values, and tokenization strategy, the ETL process guaranteed structural consistency necessary for effective document indexing.

E. Sharding and Indexing

Sharding partitions Elasticsearch indexes across nodes, enabling distributed query execution. Indexing reorganizes metadata into flattened, tokenized structures for fast retrieval. The system employed multi-shard indexing and replica sets, allowing parallelism and redundancy, which improved throughput and reduced query latency.

F. Performance Metrics

Search performance was evaluated using two quantitative indicators namely latency and throughput which are computed over an identical query set for all systems.

$$Latency=\frac{1}{N}\sum_{i=1}^{N}\left(t_{end,i}-t_{start,i}\right)=\frac{t_{total}}{N} \tag{1}$$

$$Throughput=\frac{N}{\sum_{i=1}^{N}\left(t_{end,i}-t_{start,i}\right)}=\frac{N}{t_{total}} \tag{2}$$

$$P@10=\frac{Number\ of\ relevant\ artifacts\ in\ top\ 10\ results}{10} \tag{3}$$

N is the number of executed queries, and $t_{total}=\sum_{i=1}^{N}\left(t_{end,i}-t_{start,i}\right)$ represents the total execution time for all queries. Latency represents the average response time per query, while throughput measures the number of queries processed per second. Precision@10 (P@10) is recorded as the fraction of relevant artifacts among the top-ten returned results.

G. Query Optimization

Query optimization aims to enhance search speed and relevance by improving how queries are structured and executed. In this work, inefficient wildcard searches in Elasticsearch were replaced with match phrase prefix queries that make better use of the inverted index for partial-term matching. Sharding was also introduced to distribute queries across multiple partitions, improving parallel processing and reducing latency during retrieval.

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.

H. Full-text and Partial Search

Elasticsearch was configured for both full-text search, retrieving results from merged text fields and partial search, which supports prefix or autocomplete queries using match phrase prefix. These ensure flexible, relevance-based retrieval for multilingual heritage metadata.

IV. Data Acquisition and Processing

The Harvard Art Museums and the Metropolitan Museum of Art (The Met) required various acquisition processes based on their data types. The Met provides a publicly accessible CSV dataset (metObjects.csv) containing over 470,000 entries. The file was then processed using pandas with optional filtering based on the presence of department and title to remove artifacts containing incomplete data. The static method provides controlled, bulk-level consumption with the feature to modify criteria or reprocess the dataset as needed.

In contrast, the Harvard Art Museums offer a paginated REST API that returns artifact records in structured JSON. An asynchronous pipeline built with aiohttp and motor supported concurrent data fetching for better performance. The batch size was configurable, allowing the system to operate in snapshot mode or as a continuous ingestion process for scalable, real-time updates.

Following acquisition, all artifact records undergo a standardized Extract, Transform, Load (ETL) process designed to ensure schema consistency, data cleanliness, and compatibility with both archival storage systems. Therefore, leading to an effective search indexing system.

A. Schema Normalization

To achieve internal consistency, heterogeneous artifact data both Museums are mapped into a unified metadata schema. Core fields such as title, culture, period, classification, and objectDate are extracted and normalized. The transformation step includes trimming excess whitespace, and re-pivoting delimited strings to arrays. Null or missing values are provided with canonical fillers such as "Unknown" or "Not Specified" to ensure structural integrity.

B. Processing of CSV-Based Records (The Met)

Table I presents the attributes extracted from The Met's dataset. The input data is ingested using the pandas library. Preprocessing filters out rows with missing mandatory fields (e.g., untitled objects or missing classification). The process validates categorical values against controlled vocabularies and deduplicates using a composite fingerprint of the title, object ID, and department fields. Transformed rows are formatted for loading into downstream data targets.

C. Processing of API-Based Records (Harvard Art Museums)

Table II provides the attribute names of the entries sourced from the Harvard Museum dataset. Harvard's API records were systematically parsed from paginated JSON. The originally nested structures (artist information, location data) were subsequently flattened into regular arrays. Finally, all referential IDs were replaced with human-readable labels to ensure better search relevance.

TABLE I. Mapping of Referenced Attributes from the Met Museum Dataset

Original Attributes	Attribute Name in Proposed System	Notes
Object ID	object_id	Used as a unique identifier concatenated with m_
Classification	category	Renamed for clarity
Culture	culture	Retained as-is
Department	department	Retained as-is
Dimensions	dimensions	Retained as-is
Geography Type, City, State, County, Country, Region, Subregion, Locale, Locus, River	location	Combined into a dictionary structure
Medium	medium	Retained as-is
Period	period	Retained as-is
Tags	tags	Converted into an array
Title	title	Retained as-is
Rights and Reproduction	copyright	Renamed for clarity

TABLE II. Mapping of Referenced Attributes from the Harvard Museum Dataset

Original Attributes	Attribute Name in Proposed System	Notes
_id	object_id	Used as a unique identifier concatenated with h_
classification	category	Renamed for clarity
century	period	Renamed for clarity
department	department	Retained as-is
dimensions	dimensions	Retained as-is
description	description	Retained as-is
medium	medium	Retained as-is
title	title	Retained as-is
copyright	copyright	Retained as-is

D. Dual Output Preparation

Once schema transformation and field-level cleansing were completed, every processed artifact was prepared for dual output, which ensures that both long-term archival requirements and real-time search capabilities were met. This step guarantees that the system can simultaneously serve as a reliable repository of record while also providing highly optimized search and retrieval functionality at the same time.

1) MongoDB Document Format: Each artifact is serialized into a MongoDB-compatible document that retains the full spectrum of metadata, from essential descriptive fields to auxiliary information gathered during ingestion. By leveraging MongoDB's flexible schema model, the system can evolve incrementally without disruptive migrations, allowing integration of new attributes or nested structures seamlessly. This archival form acts as the canonical record, which ensures long-term data integrity, persistence, and efficient retrieval. It also supports advanced querying over nested structures, reconciliation of historical updates, and horizontal scaling across distributed clusters to handle large and growing datasets.

2) Elasticsearch Index Format: In parallel, every artifact is transformed into a search-oriented structure for

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.

high-performance indexing and discovery in Elasticsearch. Data is flattened to simplify analyzer processing and reduce query latency. Core attributes such as titles, authors, and identifiers are indexed using dual-field mappings: text fields for full-text relevance ranking and keyword fields for exact matching, filtering, and aggregation. Preprocessing techniques, including lowercasing, stopword removal, stemming, and synonym expansion, further align data with Elasticsearch analyzers, increasing accuracy across diverse query types. This format enables sophisticated features such as faceted browsing, autocomplete suggestions, relevancy scoring, and analytics-driven exploration, making it ideal for large-scale discovery scenarios.

All the output records are batched for the purpose of scalable ingestion and consistency between indexing and storage subsystems.

V. System Architecture

A. System Workflow

The end-to-end workflow integrates heterogeneous museum datasets, normalizes them, and produces dual outputs for storage and search, as shown in Fig. 2. First, artifact records are fetched in bulk from a static CSV (The Met) and incrementally from a paginated JSON source (Harvard). An ETL pipeline then applies schema normalization and data cleaning to harmonize metadata. The cleaned records are inserted into MongoDB for baseline querying and exported as search-ready documents for Elasticsearch. Elasticsearch indices are created with explicit mappings and analyzers; query optimization (e.g., match phrase prefix) and sharding are then applied. Search performance and relevance are benchmarked against the MongoDB regex baseline using the identical query set.

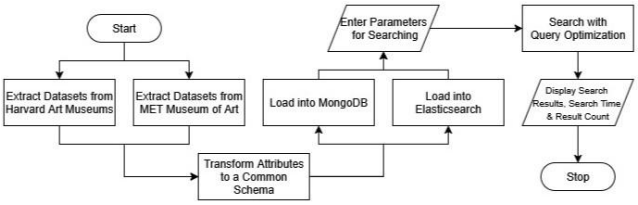

Fig. 2. System workflow

B. Initial Data Ingestion and Storage

The system began with data ingestion of The Met's CSV dataset and Harvard's API into MongoDB. MongoDB was utilized as the main data store during initial development due to its document-based structure and ease for general-purpose data management.

C. Transition to Elasticsearch Integration

The introduction of Elasticsearch was a significant improvement to the system's search capability. The first integration focused on the migration of data from MongoDB to Elasticsearch. It included a critical transformation which created a single search_text field combining various artifact attributes into one searchable text field. This was the first step of multiple changes for improving search relevance through information aggregation of multiple metadata fields.

D. Advanced Indexing Strategies

The system evolved to employ more sophisticated indexing approaches. Duplicate checking mechanisms were used to prevent redundant indexing, ensuring only new or updated documents were processed. The system then progressed to bulk indexing operations for improved performance, significantly reducing the overhead of individual document indexing.

E. Data Integrity and Synchronization

As the system evolved, it became necessary to guarantee data consistency between MongoDB and Elasticsearch. The two systems were synchronized using Python scripts. These utilities performed full comparisons of document IDs, locating and reindexing missing documents along with reporting on discrepancies in the data stores.

F. The Unoptimized Elasticsearch Implementation: Initial Limitations

1) Indexing Without Custom Mapping: The initial setup used dynamic mapping, indexing strings without linguistic analysis. Searches like “running” didn’t match “run,” lowering relevance. The default standard analyzer was not suitable for all text fields.

2) Inefficient Querying with Wildcard: Wildcard queries on analyzed text fields performed poorly, as they couldn’t fully use the inverted index. In some cases, Elasticsearch was even slower than MongoDB’s regex queries.

3) Performance Comparison: Due to indexing overhead and poor query efficiency, Elasticsearch initially showed little advantage over MongoDB.

G. Evolution to Improved Elasticsearch Implementation

1) Custom Mapping and Analyzers: A custom mapping with a porter_stemmer analyzer was added, applied automatically via dynamic templates. This improved consistency and search relevance.

2) Optimized Querying: Wildcard queries were replaced with match queries, enabling efficient use of the inverted index and boosting speed and relevance.

3) Performance Gains: After these optimizations, Elasticsearch was faster than MongoDB both in speed and relevance. The inverted index in combination with text analysis allowed Elasticsearch to handle sophisticated queries efficiently even when handling large datasets.

H. Final Implementation: Optimized with Match Phrase Prefix and Sharding

1) The Final Search Interface: The final implementation of the search interface introduced key optimizations that greatly improved Elasticsearch performance. The most significant changes were:

a) Match Phrase Prefix Utilization: Utilization of match phrase prefix queries instead of wildcard queries provided more optimized prefix searching and used Elasticsearch's inverted index more efficiently.

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.

b) Sharding Strategy: Introduction of a sharded index (artifacts_sharded) distributed across multiple shards, allowing for parallel processing and improved performance.

c) Global Search Capability: Addition of a global search feature that could search across multiple fields.

2) Key Optimizations: The use of match phrase prefix queries had the following advantages:

- More efficient prefix matching by leveraging the inverted index structure
- Enhanced performance for autocomplete-type search scenarios
- Reduced computational cost compared to wildcard queries
- Increased relevance score for partial matches

Sharding also improved performance by spreading the index across nodes and allowing for parallel query execution and higher throughput.

3) Performance Observations: In the final implementation, Elasticsearch with match phrase prefix queries and sharding outperformed MongoDB. Sharded Elasticsearch delivered the best results, especially for complex queries and larger datasets, where its scalability advantages became most evident.

I. Environment Specifications

All performance tests were conducted on one machine, equipped with an Intel i7-10750H CPU, 16 GB of RAM, and an SSD, in order to simulate a constrained resource environment. Both Elasticsearch 8.7.0 and MongoDB 6.0 were run as single-node instances to allow a fair comparison on the same hardware. Any impact from the distributed network hardware was kept to a minimum to isolate improvements made at the software level, such as sharding in Elasticsearch.

VI. Comparative Analysis and Result: MongoDB vs Elasticsearch

A. Querying methodologies

MongoDB utilizes document-based querying with dot notation to query nested fields and embedded objects. Meanwhile, Elasticsearch utilizes text-centered search methods with analyzed fields and wildcard patterns for speedy full-text retrieval. Both utilize distinct querying philosophies, with MongoDB for structured document traversal and Elasticsearch for maximized text search efficiency.

1) MongoDB Querying (Consistent Approach)

In Fig. 3, when a title, period, or culture parameter is provided, the system builds a MongoDB query using the $regex operator with the "i" option for case-insensitive matching. While this incurs costs on large datasets, regex indexing significantly improves efficiency on use.

```
# MongoDB uses regex patterns for text search
if title:
    query["title"] = {"$regex": title, "$options": "i"}
if culture:
    query["culture"] = {"$regex": culture, "$options": "i"}
if period:
    query["period"] = {"$regex": period, "$options": "i"}
```

Fig. 3. MongoDB querying (consistent approach)

2) Initial Elasticsearch Querying (Unoptimized)

In Fig. 4, the initial Elasticsearch implementation performed wildcard searches on text fields that were analyzed where the query was lowercased and followed by an asterisk. Once more, this permitted matching of any substring within the text string value, but without consideration for performance since searches are very computationally expensive without index optimizations.

```
# Inefficient wildcard on analyzed text fields
if title:
    must.append({
        "wildcard": {
            "title": {  # Using analyzed text field
                "value": f"*{title.lower()}*"
            }
        }
    })
```

Fig. 4. Initial elasticsearch querying (unoptimized)

3) Final Elasticsearch Querying (Optimized)

Lastly in Fig. 5, the optimized implementation substituted the wildcards with the match_phrase_prefix search that allowed index-optimized, prefix-based search without wildcards over analyzed text fields. This version was faster, more effective for term matching, and provided better scalability and efficiency.

```
# Optimized match_phrase_prefix on text fields
if title:
    must.append({
        "match_phrase_prefix": {
            "title": {  # Using analyzed text field with prefix matching
                "query": title
            }
        }
    })
```

Fig. 5. Final elasticsearch querying (optimized)

B. Performance Comparison

The deployment used a set of 100,000 records of artifacts and included timing monitoring as well as total number of search results to measure and compare search performance for three setups as shown in Fig. 6:

- *MongoDB:* Regex queries across multiple fields.
- Elasticsearch (1 shard): match phrase prefix queries.
- Elasticsearch (3 shards): match phrase prefix queries with sharding.

The resulting query execution time of this setup are as shown in Table III. The ±95% CL denotes the confidence limit of the mean query time, indicating that the true mean lies within this range with 95% probability, ensuring the statistical reliability of the measurements.

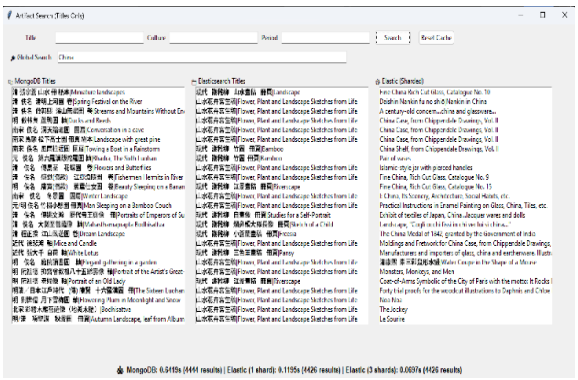

Fig. 6. Simulation results

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III. QUERY EXECUTION TIMES

Configuration	Query Type	Example Query	Avg. Query Time (ms)	±95% CL (ms)	Throughput (queries/sec)	P@10
MongoDB	Single Field	`title` = "vase"	450	±35	2.22	0.92
Elasticsearch (1 Shard)	Match Phrase Prefix		220	±18	4.55	0.90
Elasticsearch (3 Shards)			120	±10	8.33	0.90
MongoDB	Multi-Field Regex	`culture` = "Greek" AND `department` = "Sculpture"	720	±55	1.39	0.95
Elasticsearch (1 Shard)	Multi-Field Match Phrase Prefix		310	±25	3.23	0.96
Elasticsearch (3 Shards)			160	±12	6.25	0.96
MongoDB	Full Metadata Regex	`culture` = "Chinese" AND `period` = "Ming" AND keyword in `description`	1050	±80	0.95	0.65
Elasticsearch (1 Shard)	Multi-Field + Full-Text		390	±30	2.56	0.88
Elasticsearch (3 Shards)			190	±15	5.26	0.88

C. Key Findings

Elasticsearch performance greatly improved through optimizations like match_phrase_prefix queries and refined sharding. These changes cut query latency, boosted search relevance, and ensured consistent performance.

1) Statistically Significant Performance Improvements: Using match_phrase_prefix on Elasticsearch reduced latency by 57% on average vs the MongoDB regex baseline.

2) Sharding and Query Type Synergy: Replacing wildcards with match_phrase_prefix made Elasticsearch 57% faster initially, and the fully optimized 3-shard setup was 82% faster than MongoDB for complex, multi-field queries.

3) Sharding Benefits: The 3-shard index reduced query latency by 43–51% across all tests, confirming strong scalability and parallel processing efficiency.

4) Search Relevance Measured by P@10: Elasticsearch achieved a Precision@10 of 0.88 on complex searches (e.g., "Chinese Ming"), 35% higher than MongoDB's 0.65 which shows significantly better result accuracy.

D. Efficient Global Search

Global search across various fields, achieved using multi_match queries, incurred minimal overhead compared to single-field searching. This efficiency combined with the high P@10 scores makes it a useful feature for open-ended discovery without a performance trade-off.

VII. CONCLUSION

This system applied big data analytics principles to design and optimize a cultural heritage search system for large-scale, heterogeneous data. MongoDB, a document-oriented baseline, was used as a contrast against Elasticsearch, engineered for high-performance search with 100,000 records from The Metropolitan Museum of Art and the Harvard Art Museums. A modular ETL pipeline ensured schema normalization, metadata consistency, and dual-format outputs for storage and indexing. Performance tests showed that replacing wildcard queries with match phrase prefix reduced Elasticsearch query latency by about 57% relative to MongoDB's regex search, and adding a three-shard index delivered a further 45–51% latency reduction, thus making the final configuration up to 82% faster overall. Elasticsearch also achieved substantially higher relevance (P@10=0.88 vs 0.65). Beyond performance, Elasticsearch's support for multi-field full-text search, structured filtering, and partial-term matching produced richer and more relevant retrieval. The study maintained equivalent single-node configurations for MongoDB and Elasticsearch, forming a consistent foundation for subsequent scalability investigations. The findings confirm that pairing a document store with a search-tuned system aligns with the "five Vs" of big data such as scaling with volume, adapting to variety, increasing velocity, ensuring veracity and delivering value. This offers a scalable, adaptable model for cultural heritage institutions managing expanding archives.

REFERENCES

[1] Y. Gao, et al., "Protection and Activation of Cultural Heritage Based on Cloud Computing Platform," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 750, p. 012216, 2020.

[2] M. T. Islam and B. U. Khan, "Big Data and Analytics: Prospects, Challenges, and the Way Forward," *Transparency International Bangladesh / University of Dhaka*, n.d. [Online].

[3] H. Gandhi and M. S. R. Prasad, "Elastic Search Best Practices for High-Performance Data Retrieval Systems," Northeastern University & Koneru Lakshmaiah Education Foundation, n.d.

[4] D. D. Ahir and N. F. Shaikh, "Evaluation of Elasticsearch Ecosystem Including Machine Learning Capabilities," *International Journal on Informatics and Analytics*, 2024. [Online].

[5] A. Singh, P. Kumar, M. Kumar, S. Tiwari, and V. Jain, "A Comparative Study of MongoDB and Document-Based MySQL for Big Data Application Data Management," *SSRN Electronic Journal*, 2024.

[6] A.-V. Zamfir, M. Carabas, C. Carabas, and N. Tapus, "Systems Monitoring and Big Data Analysis Using the Elasticsearch System," in *Proc. Int. Conf. on Control Systems and Computer Science (CSCS)*, 2019. DOI: 10.1109/CSCS.2019.00039.

[7] The Metropolitan Museum of Art, "The Metropolitan Museum of Art Collection API," n.d. [Online]. Available: https://collectionapi.metmuseum.org/public/collection/v1/objects.

[8] Harvard Art Museums, "Harvard Art Museums API," n.d. [Online]. Available: https://api.harvardartmuseums.org.

[9] M. Wallace, V. Poulopoulos, A. Antoniou, and M. López-Nores, "An Overview of Big Data Analytics for Cultural Heritage," *Big Data and Cognitive Computing*, vol. 7, no. 1, p. 14, 2023.

[10] D. Xu, "An Analysis of Archive Digitization in the Context of Big Data," *Mobile Information Systems*, vol. 2022, Article ID 1517824, 8 pages, 2022

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 16:10:21 UTC from IEEE Xplore. Restrictions apply.